自然语言驱动排障
让每个判断有据可查

围绕同一个对象，把告警、指标、日志
与依赖关系连起来，支持持续追问。



选定问数或排障模式，确认对象与时间，再开始调查。

1 **提问** 自然语言 → 2 **确认** 对象与时间 → 3 **取证** 受控查询 → 4 **判断** 依据与缺口 → 5 **验证** 处理后再检查

六项技术优势

让提问更方便，让依据与处理结果更清楚。

01

找准问题对象

核对集群、资源标识与时间，
区分同名资源和重建对象。

排查目标更清楚

02

自然语言持续追问

沿用已确认的上下文，
有歧义时先请用户确认。

减少重复输入

03

判断依据可以核对

展示事实、来源与引用，
说明反证和证据缺口。

知道为何这样判断

04

关联跨层故障线索

串联应用、节点、网络、
存储与硬件的可信关系。

更容易检查共同依赖

05

调查过程能够延续

保留每轮问题、查询步骤
和分析版本，方便交接。

调查来龙去脉可回看

06

处置结果再次验证

本人确认操作后，读取
新的观测检查是否恢复。

分清执行成功与恢复

问数查询 · 应用排障 · 节点与存储分析 · 硬件线索关联

面向 Kubernetes 与物理基础设施的非虚拟化智能运维

一次排查，从问题走到结果

以应用反复重启为例，看自然语言、真实证据和恢复检查如何配合。

排障交互示意

已选定应用对象

用户	排查这个应用最近30分钟的重启问题。
平台	核对身份与时间 → 查询证据 → 说明依据与缺口
追问	再检查它所在的节点，看看是否相关。
平台	沿真实关系继续只读检查，补充事实并保留本轮记录。

示意流程，实际原因以当次证据为准。

技术如何支撑这些能力

模型组织解释，平台负责把关查询、权限和证据。

01	交互工作台	提问、确认对象与时间、查看证据、持续追问
02	任务控制	校验身份与范围，保存进度、调用和分析版本
03	规则与模型	规则核对条件，模型解释事实、组织候选原因
04	受控数据工具	通过 MCP 查询获准来源，校验回答引用

资源 | 指标 | 日志 | 事件 | 网络 | 硬件

技术底座 React / TypeScript · Go · Python / HolmesGPT · MCP

从建议走向可核对的处理结果

本人输入并确认命令 → 查看执行回执 → 用新观测验证恢复

能力按项目验证后开放，排障、追问与故障链分别验证模型任务、工具和来源。跨层关系用于排查，因果判断仍需证据；日志与网络结论受来源覆盖限制。虚拟化、故障预测与完整链路追踪不在本彩页范围。